
Libiio c/c++使用例子

Rev.1.0



1. 版本记录

版本	时间	描述
Rev. 1.0	2022.11.8	第一版发布

2.免责声明：

产品中所提供的程序源代码、软件、资料文档等，微相科技不提供任何类型的担保；不论是明确的，还是隐含的，包括但不限于合适特定用途的保证，全部的风险，由使用者来承担。

前 言

Libiio 是由 Analog Devices 开发的一个库，用于简化与 Linux 工业 I/O (IIO) 设备接口的软件开发。该库抽象了硬件的低级细节，并提供了一个简单而完整的编程接口，可用于高级项目。

这里，我们通过编译 Libiio 提供的一个例子，分别将其运行在主机上和嵌入式平台上。该例子是通过 c/c++接口来配置射频端，并填充获取发送接收 IQ 数据流。该文档适合有编程基础的人观看，涉及到的内容有 c/c++，交叉编译等。本章所介绍内容均运行在 Linux 平台下 (Ubuntu)。

第 1 章 环境配置

1.1 libiio 库下载安装

官方将 libiio 库的源码放在 [github 仓库](#)上，这里我们通过编译源码的方式来安装 libiio 库。

1.1.1 依赖安装

在 ubuntu 下运行以下命令：

```
sudo apt-get install libxml2 libxml2-dev bison flex libcdk5-dev cmake  
sudo apt-get install libaio-dev libusb-1.0-0-dev libserialport-dev libxml2-  
dev libavahi-client-dev doxygen graphviz
```

1.1.2 下载编译安装 libiio

在 ubuntu 下运行以下命令：

```
git clone https://github.com/pcercuei/libini.git  
cd libini  
mkdir build && cd build && cmake ../ && make && sudo make install
```

```
git clone https://github.com/analogdevicesinc/libiio.git  
mkdir build && cd build && cmake ../ && make && sudo make install
```

1.2 libiio 测试

依次执行上述步骤后，用户可以通过 iio 命令来测试是否安装成功。

首先给 antsdr 系列板子上电，并确保运行固件为 iio 系列固件。

```
iio_info -u "ip:192.168.1.10"
```

```

[cc@jce:~]$ iio_info -u "ip:192.168.1.10"
Library version: 0.24 (git tag: c4498c2)
Compiled with backends: local xml tp usb
IIO context created with network backend.
Backend version: 0.21 (git tag: v0.21 )
Backend description string: 192.168.1.10 Linux (none) 5.10.0-97864-g22b4eccaa8d1-dirty #20 SMP PREEMPT Sat Oct 8 19:34:30 CST 2022 armv7l
IIO context has 9 attributes:
  hw_model: Analog Devices PlutoSDR Rev.C (Z7010-AD9364)
  hw_model_variant: 1
  hw_serial: db008c3783185036
  fw_version: e310_v1.2-3-g91c7-dirty
  ad9361-phy_xo_correction: 40000000
  ad9361-phy_model: ad9364
  local_kernel: 5.10.0-97864-g22b4eccaa8d1-dirty
  uri: ip:192.168.1.10
  tp_ip_addr: 192.168.1.10
IIO context has 4 devices:
  iio:device0: ad9361-phy
    9 channels found:
      altvoltage1: TX_LO (output)
        8 channel-specific attributes found:
          attr 0: external value: 0
          attr 1: fastlock_load value: 0
          attr 2: fastlock_recall ERROR: Invalid argument (22)
          attr 3: fastlock_save value: 0 84,84,84,84,84,84,84,84,84,84,84,84,84,84,84,84,84,84
          attr 4: fastlock_store value: 0
          attr 5: frequency value: 2450000000
          attr 6: frequency_available value: [46875001 1 6000000000]
          attr 7: powerdown value: 0
      voltage0: (input)
        15 channel-specific attributes found:
          attr 0: bb_dc_offset_tracking_en value: 1
          attr 1: filter_fir_en value: 0
          attr 2: gain_control_mode value: slow_attack
          attr 3: gain_control_mode_available value: manual fast_attack slow_attack hybrid
          attr 4: hardwaregain value: 71.000000 dB
          attr 5: hardwaregain_available value: [-3 1 71]
          attr 6: quadrature_tracking_en value: 1
          attr 7: rf_bandwidth value: 18000000
          attr 8: rf_bandwidth_available value: [200000 1 56000000]
          attr 9: rf_dc_offset_tracking_en value: 1
          attr 10: rf_port_select value: A_BALANCED
          attr 11: rf_port_select_available value: A_BALANCED B_BALANCED C_BALANCED A_N_A_P B_N_B_P C_N_C_P TX_MONITOR1 TX_MONITOR2 TX_MONITOR1_2
          attr 12: rssi value: 114.50 dB

```

上图说明已在主机上成功安装 libiio 库并与设备连接。

第 2 章 运行例子编译运行

2.1 源码

Microphase 已经为用户提供了源码，这里我们需要编译 `ad9361-iostream.c` 文件并运行。首先新建一个文件夹（`libiio_example`），并在该文件夹下新建名为 `arm` 和 `host` 的文件夹。将 `ad9361-iostream.c` 文件拷贝到上述文件夹下，如图：

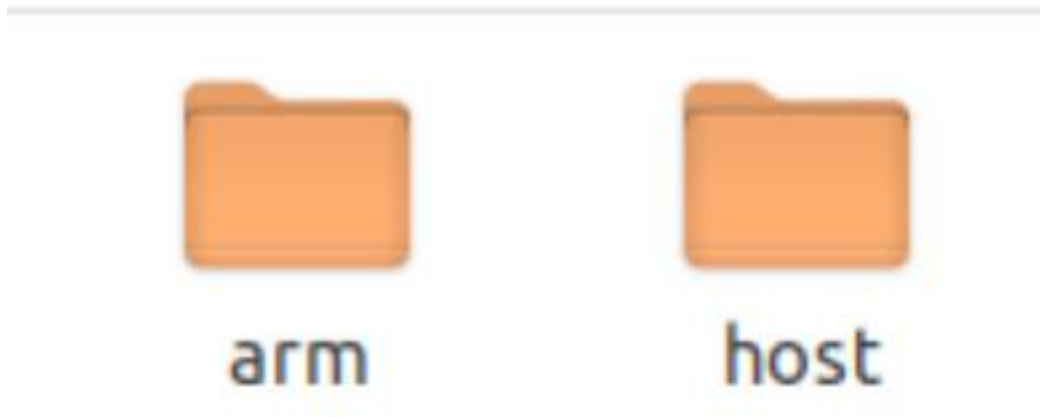


图 2-1

2.2 添加 CMakeLists 文件

接下来需要提供 `Cmakeists` 文件，这里 Microphase 为用户已经写好，用于需要将其分别放在不同的文件夹目录下，如图（`host` 和 `arm` 的 `Cmakelist` 并不相同）：

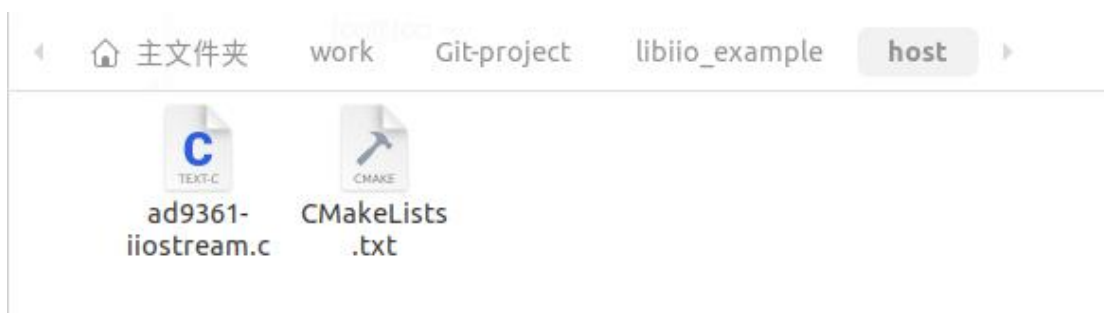


图 2-2

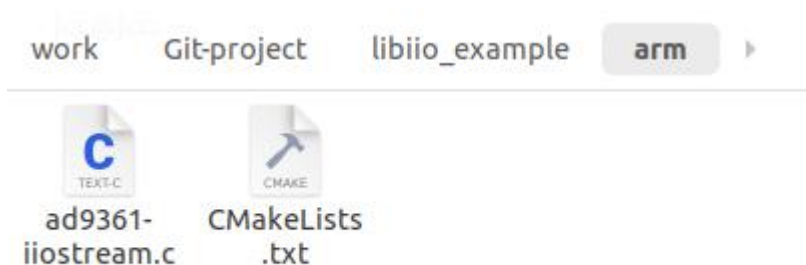


图 2-3

2.3 编译运行

2.3.1 host 主机

在 host 目录下运行以下命令：

```
mkdir build && cd build  
cmake ..  
make
```

接下来可以在 build 文件夹下看见可执行文件如图：

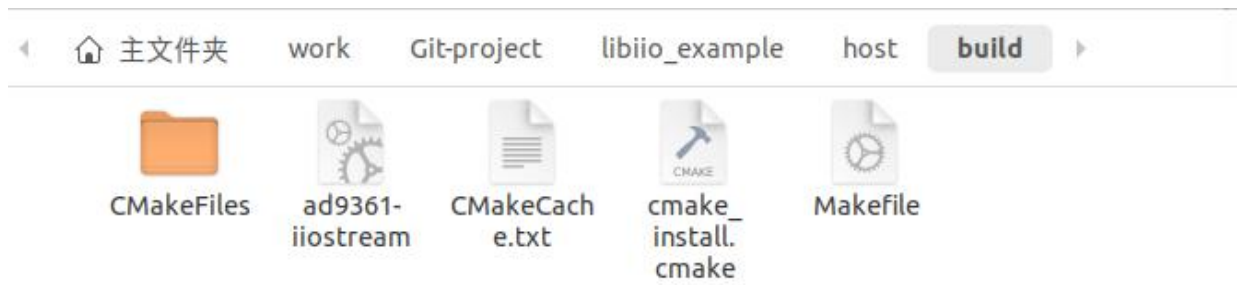


图 2-4

运行命令 `./ad9361-iostream` 即可看见以下结果如图：

```
jcc@jcc:~/work/Git-project/libiio_example/host/build$ ./ad9361-iostream
* Acquiring IIO context
* Acquiring AD9361 streaming devices
* Configuring AD9361 for streaming
* Acquiring AD9361 phy channel 0
* Acquiring AD9361 RX lo channel
* Acquiring AD9361 phy channel 0
* Acquiring AD9361 TX lo channel
* Initializing AD9361 IIO streaming channels
* Enabling IIO streaming channels
* Creating non-cyclic IIO buffers with 1 MiS
* Starting IIO streaming (press CTRL+C to cancel)
  RX    1.05 MSmp, TX    1.05 MSmp
  RX    2.10 MSmp, TX    2.10 MSmp
  RX    3.15 MSmp, TX    3.15 MSmp
  RX    4.19 MSmp, TX    4.19 MSmp
  RX    5.24 MSmp, TX    5.24 MSmp
  RX    6.29 MSmp, TX    6.29 MSmp
  RX    7.34 MSmp, TX    7.34 MSmp
  RX    8.39 MSmp, TX    8.39 MSmp
  RX    9.44 MSmp, TX    9.44 MSmp
  RX   10.49 MSmp, TX   10.49 MSmp
  RX   11.53 MSmp, TX   11.53 MSmp
  RX   12.58 MSmp, TX   12.58 MSmp
  RX   13.63 MSmp, TX   13.63 MSmp
  RX   14.68 MSmp, TX   14.68 MSmp
  RX   15.73 MSmp, TX   15.73 MSmp
  RX   16.78 MSmp, TX   16.78 MSmp
  RX   17.83 MSmp, TX   17.83 MSmp
^CWaiting for process to finish...
  RX   18.87 MSmp, TX   18.87 MSmp
* Destroying buffers
* Disabling streaming channels
* Destroying context
```

2.3.2 arm 主机

由于在主机上编译 arm，因此这里涉及到了交叉编译的知识，这里我们需要 arm 编译工具，笔者使用了 Xilinx SDK 带的 32 位编译工具。同时，我们还需

要编译 antsdrr 固件的源码，关于固件源码的编译方法需要用户更具以下[文档](#)自行操作。编译固件的时候要注意用户使用的设备（E310 或 E200）。

编译完固件后，修改 arm 目录下的 cmakefile，如图：

```
set(CMAKE_C_COMPILER "/tools/Xilinx/SDK/2019.1/gnu/aarch32/bin/gcc-arm-linux-gnueabi/bin/arm-linux-gnueabi-gcc")
set(CMAKE_CXX_COMPILER "/tools/Xilinx/SDK/2019.1/gnu/aarch32/bin/gcc-arm-linux-gnueabi/bin/arm-linux-gnueabi-g++")
```

图 2-5

第 4 第 5 行设置编译工具链路径，以及第 18 行设置固件编译目录如图：

```
set(MYSYSROOT /home/jcc/work/Git-project/antsdr-fw/buildroot/output/staging/)
set(CMAKE_FIND_ROOT_PATH "${MYSYSROOT}")
```

图 2-6

设置完成后在 arm 目录运行以下命令：

```
mkdir build && cd build
cmake ..
make
```

编译完成后，我们可以查看可执行文件信息：

运行命令：file ad9361-iiostream，结果如图：

```
|ad9361-iiostream: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux-armhf.so.3, for GNU/Linux
|3.2.0, BuildID[sha1]=9ec236ad3cd66475d0adf3d83d6f192482bbaa49, not stripped
```

图 2-7

发现是 32 位可执行文件，编译成功。

使用 scp 命令将其发送到板子上运行，scp ad9361-iiostream
[root@192.168.1.10:~](#)

接下来通过串口进入板子系统可以看见以后文件：

2.4 总结

以上文档重点讲述了如何编译 libiio 接口程序并运行在主机和 arm 上，关于 libiio api 接口需要读者自行查询 [api 文档](#)。